

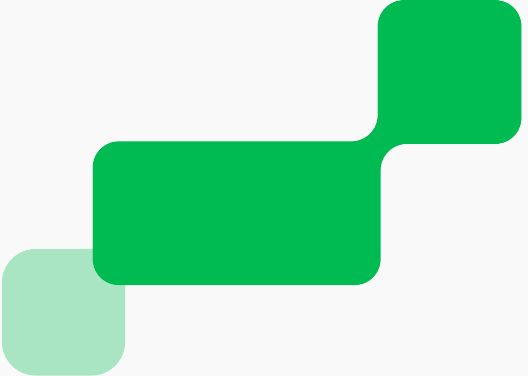
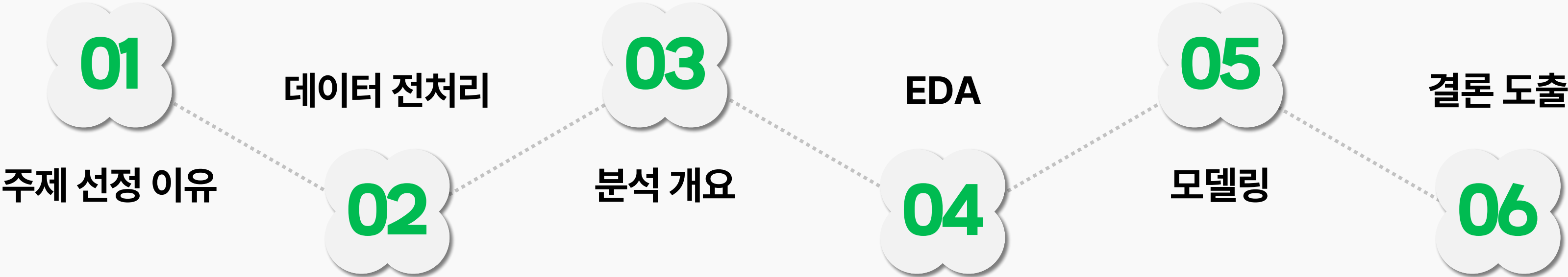
Kuggle

뉴욕 중심지 가성비 숙소 추천하기

1차 프로젝트 2조

14기 고성수, 김유빈, 이지훈, 이하린

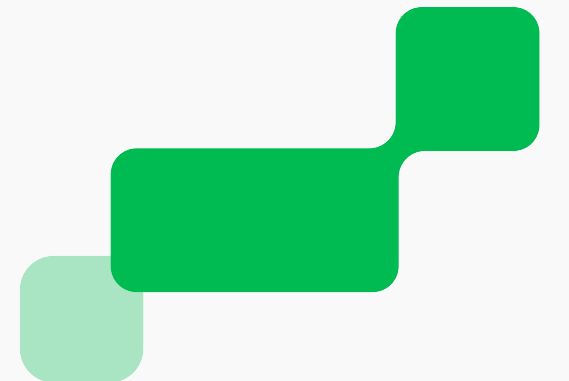




주제 선정 이유



- 호텔 수는 많지만, 가격에 비해 비교적 상태가 좋은 숙소에 머물고 싶음.
- 숙소 가격에 영향을 미치는 요인을 분석하여, 조건 대비 상대적으로 합리적인 가격의 숙소를 찾고자 함.



분석 개요: 변수 소개

Columns

- name
- host_id
- host_name
- neighbourhood_group
- neighbourhood
- latitude
- longitude
- room_type
- price
- minimum_nights
- number_of_reviews
- last_review
- reviews_per_month
- calculated_host_listings_count
- availability_365

데이터 전처리

1. Manhattan만 선택한 이유

- 대다수의 뉴욕 여행객이 먼저 고려하는 지역은 맨해튼이며, '가성비 숙소 추천'이라는 서비스 타겟에 가장 부합하는 지역이라고 판단.
- 뉴욕 내 지역마다 숙소 가격 편차가 큼. ex) 서울의 숙소와 경기도의 숙소 가격 차이는 다르게 둘 수 밖에 없음.

2. 리뷰 없는 숙소를 제외한 이유

- 운영 여부가 불분명한 '유령 숙소'를 걸러내어 추천의 정확도를 높이고 최소한의 이용 후기가 있는 숙소만 선별하여 '실제 투숙 환경'이 확인된 곳만 추천하였음.

1. 지역 필터링

```
df = df[df['neighbourhood_group'] == 'Manhattan']
```

2. 리뷰 없는 숙소 제거

```
df = df[df['number_of_reviews'] > 0]
```

3. 날짜 변환 (수치형)

```
df['last_review'] = pd.to_datetime(df['last_review'])  
df['days_since_last_review'] = (df['last_review'].max() - df['last_review']).dt.days
```

4. 타겟 로그 변환 (가격 불균형 해소)

```
y = np.log1p(df['price'])
```

EDA

자료 불러오기

```
import pandas as pd
data_path = 'manhattan_cleaned2.csv'
df = pd.read_csv(data_path)
```

각 변수의 대푯값(평균, 중앙값, 최빈값) 확인

```
print("평균:\n", df.mean(numeric_only=True))
print("중앙값:\n", df.median(numeric_only=True))
print("최빈값:\n", df.mode().iloc[0])
```

▼ Output

평균:		중앙값:		최빈값:	
host_id	7.465649e+07	host_id	3.285374e+07	name	Harlem Gem
latitude	4.076628e+01	latitude	4.076164e+01	host_id	219517861.0
longitude	-7.397395e+01	longitude	-7.398019e+01	host_name	Sonder (NYC)
price	1.944778e+02	price	1.500000e+02	neighbourhood_group	Manhattan
minimum_nights	8.084736e+00	minimum_nights	3.000000e+00	neighbourhood	Harlem
number_of_reviews	3.658361e+01	number_of_reviews	1.400000e+01	latitude	40.76035
reviews_per_month	1.732010e+00	reviews_per_month	1.120000e+00	longitude	-73.99253
calculated_host_listings_count	1.282828e+01	calculated_host_listings_count	1.000000e+00	room_type	Entire home/apt
availability_365	1.672306e+02	availability_365	1.590000e+02	price	150.0
				minimum_nights	1.0
				number_of_reviews	1.0
				last_review	2019-06-23
				reviews_per_month	1.0
				calculated_host_listings_count	1.0
				availability_365	365.0

자료 확인하기

```
df.head()
```

행, 열 개수 확인 - 데이터의 크기 파악

```
print(f"DataFrame의 행과 열: {df.shape}")
```

▼ Output

	name	host_id	host_name	neighbourhood_group	neighbourhood	latitude	longitude	room_type	price	minimum_nights	number_of_reviews	last_review	reviews_per_month	calculated_host_listings_count	availability_365
0	Skylit Midtown Castle	2845	Jennifer	Manhattan	Midtown	40.75362	-73.98377	Entire home/apt	225	1	45	2019-05-21	0.38	2	355
1	Large Cozy 1 BR Apartment In Midtown East	7322	Chris	Manhattan	Murray Hill	40.74767	-73.97500	Entire home/apt	200	3	74	2019-06-22	0.59	1	129
2	Large Furnished Room Near B'way	8967	Shunichi	Manhattan	Hell's Kitchen	40.76489	-73.98493	Private room	79	2	430	2019-06-24	3.47	1	220
3	Cute & Cozy Lower East Side 1 bdrm	7549	Ben	Manhattan	Chinatown	40.71344	-73.99037	Entire home/apt	150	1	160	2019-06-09	1.33	4	188
4	Beautiful 1br on Upper West Side	7702	Lena	Manhattan	Upper West Side	40.80316	-73.96545	Entire home/apt	135	5	53	2019-06-22	0.43	1	6

DataFrame의 행과 열: (10692, 15)

자료 상세 분석

```
df.info(memory_usage='deep')
```

숫자형 변수 확인 - 자료 내 수치형 변수 확인

```
df.select_dtypes(include=['number'])
```

▼ Output

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10692 entries, 0 to 10691
Data columns (total 15 columns):
#   Column              Non-Null Count  Dtype
---  -
0   name                10692 non-null object
1   host_id             10692 non-null int64
2   host_name           10690 non-null object
3   neighbourhood_group 10692 non-null object
4   neighbourhood        10692 non-null object
5   latitude            10692 non-null float64
6   longitude            10692 non-null float64
7   room_type           10692 non-null object
8   price               10692 non-null int64
9   minimum_nights      10692 non-null int64
10  number_of_reviews   10692 non-null int64
11  last_review         10692 non-null object
12  reviews_per_month   10692 non-null float64
13  calculated_host_listings_count 10692 non-null int64
14  availability_365     10692 non-null int64
dtypes: float64(3), int64(6), object(6)
```

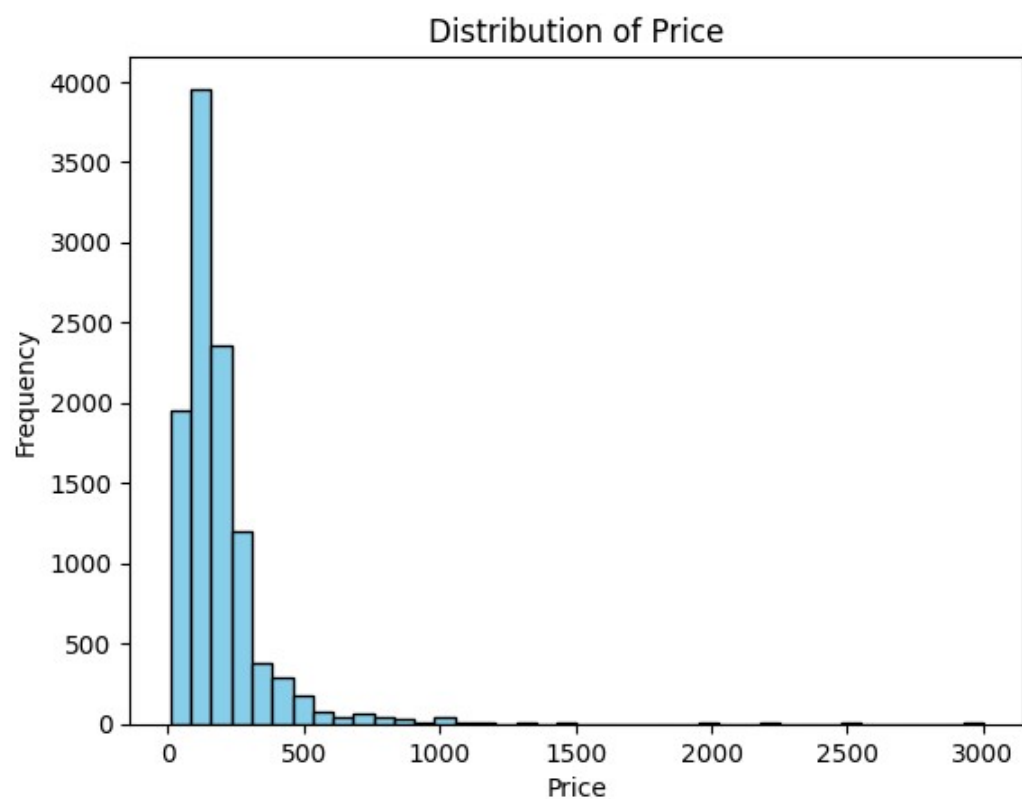
	host_id	latitude	longitude	price	minimum_nights	number_of_reviews	reviews_per_month	calculated_host_listings_count	availability_365
0	2845	40.75362	-73.98377	225	1	45	0.38	2	355
1	7322	40.74767	-73.97500	200	3	74	0.59	1	129
2	8967	40.76489	-73.98493	79	2	430	3.47	1	220
3	7549	40.71344	-73.99037	150	1	160	1.33	4	188
4	7702	40.80316	-73.96545	135	5	53	0.43	1	6
...	...	...	...	...	...	...	...	...	...
10687	52917571	40.75286	-73.99297	120	2	1	1.00	1	7
10688	269870940	40.70603	-74.01084	75	1	1	1.00	1	181
10689	273361532	40.80281	-73.96550	110	3	2	2.00	1	15
10690	33917435	40.71825	-73.99019	150	4	1	1.00	1	13
10691	83554966	40.78099	-73.95366	129	1	1	1.00	1	147

10692 rows × 9 columns

수치형 변수 분포 시각화

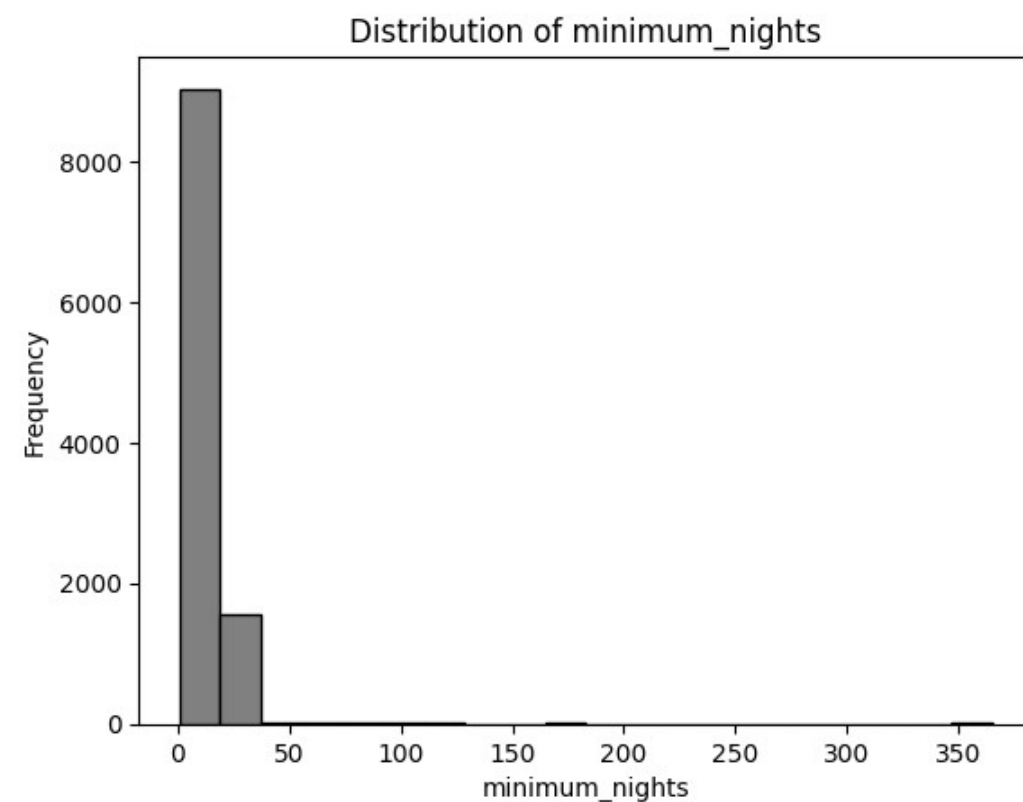
가격 분포 히스토그램

```
import matplotlib.pyplot as plt
plt.hist(df[df['price'] < 3500]['price'], bins=40, edgecolor='black',
        color='skyblue')
plt.xlabel('Price')
plt.ylabel('Frequency')
plt.title('Distribution of Price')
plt.show()
```



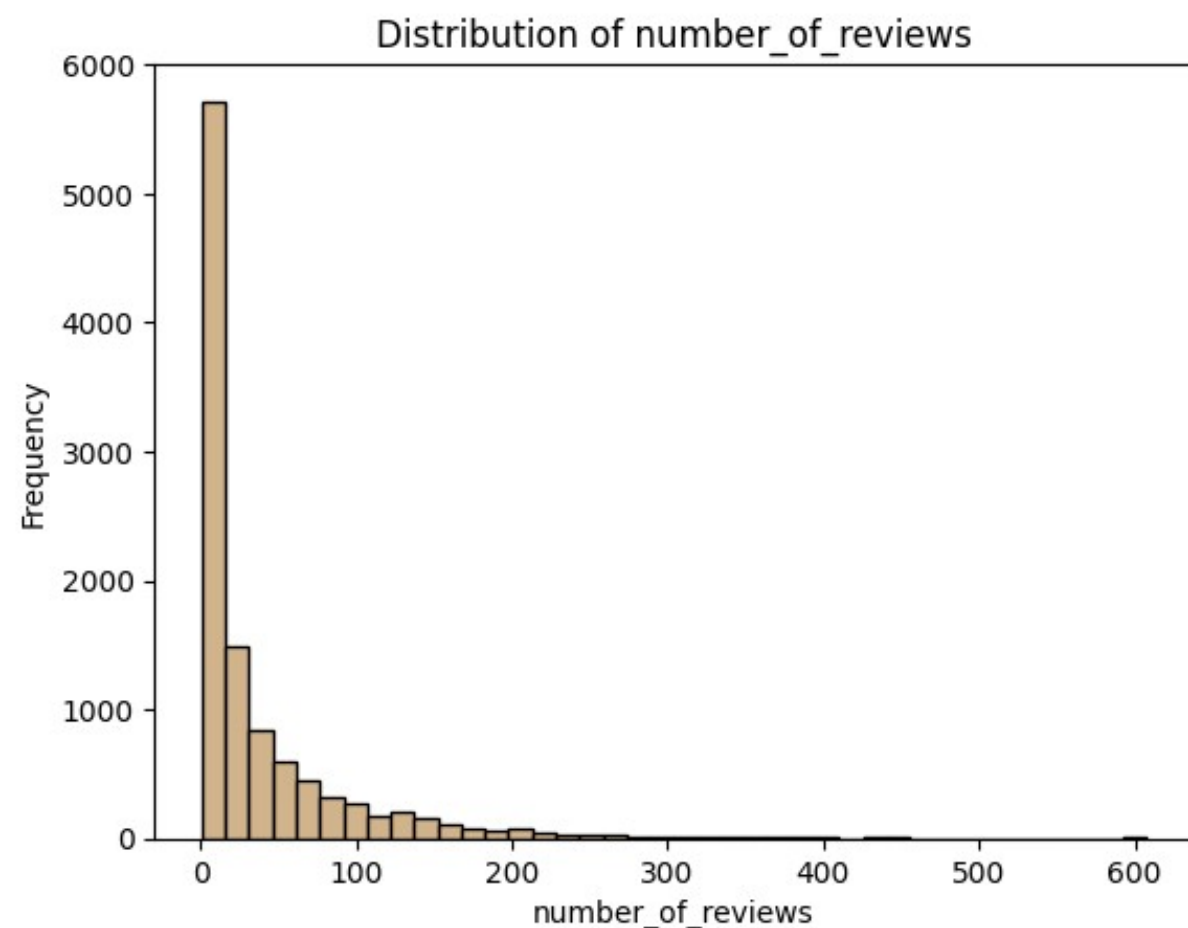
최소 숙박 일수 히스토그램

```
plt.hist(df[df['minimum_nights'] < 500]['minimum_nights'], bins=20,
        edgecolor='black', color='grey')
plt.xlabel('minimum_nights')
plt.ylabel('Frequency')
plt.title('Distribution of minimum_nights')
plt.show()
```



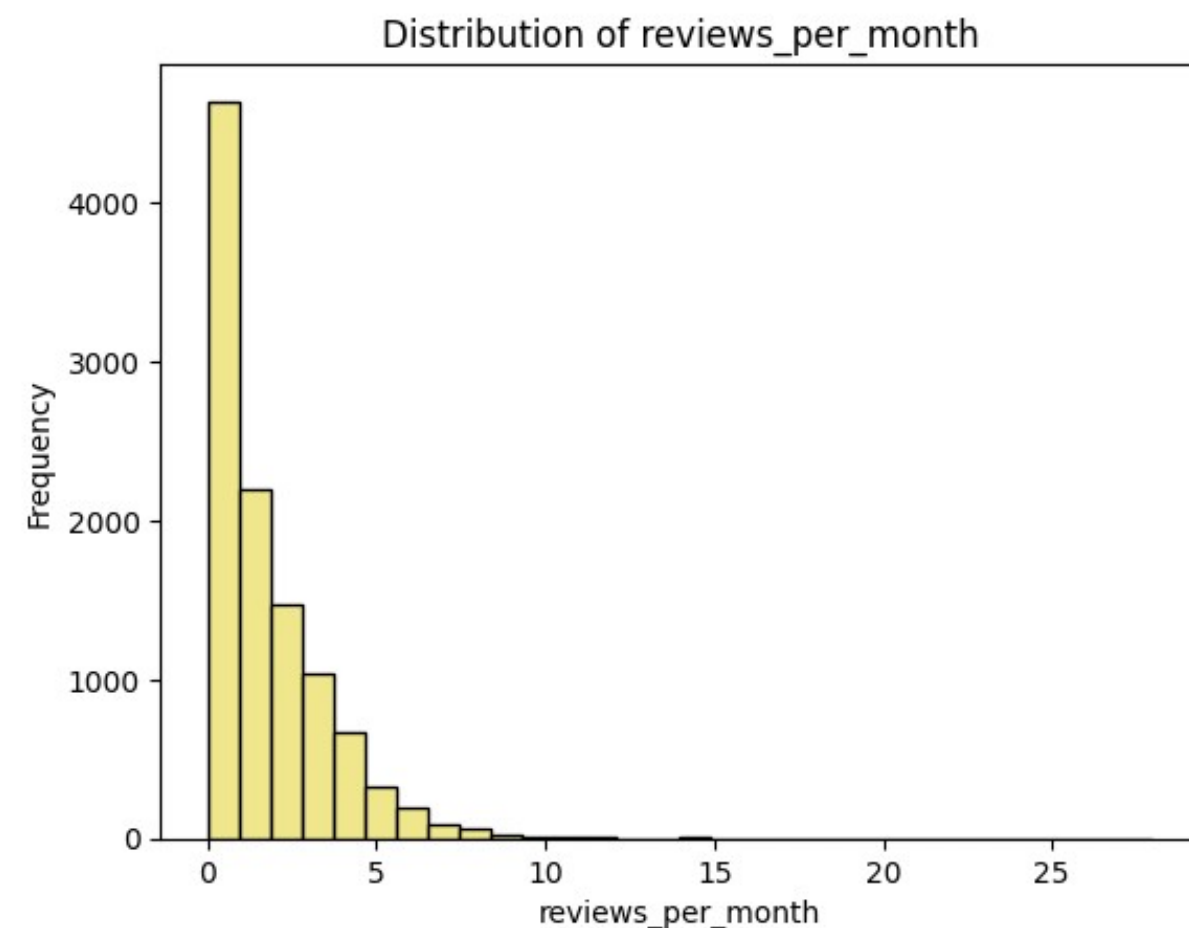
리뷰 개수 분포 히스토그램

```
plt.hist(df['number_of_reviews'], bins=40,  
         edgecolor='black', color='tan')  
plt.xlabel('number_of_reviews')  
plt.ylabel('Frequency')  
plt.title('Distribution of number_of_reviews')  
plt.show()
```



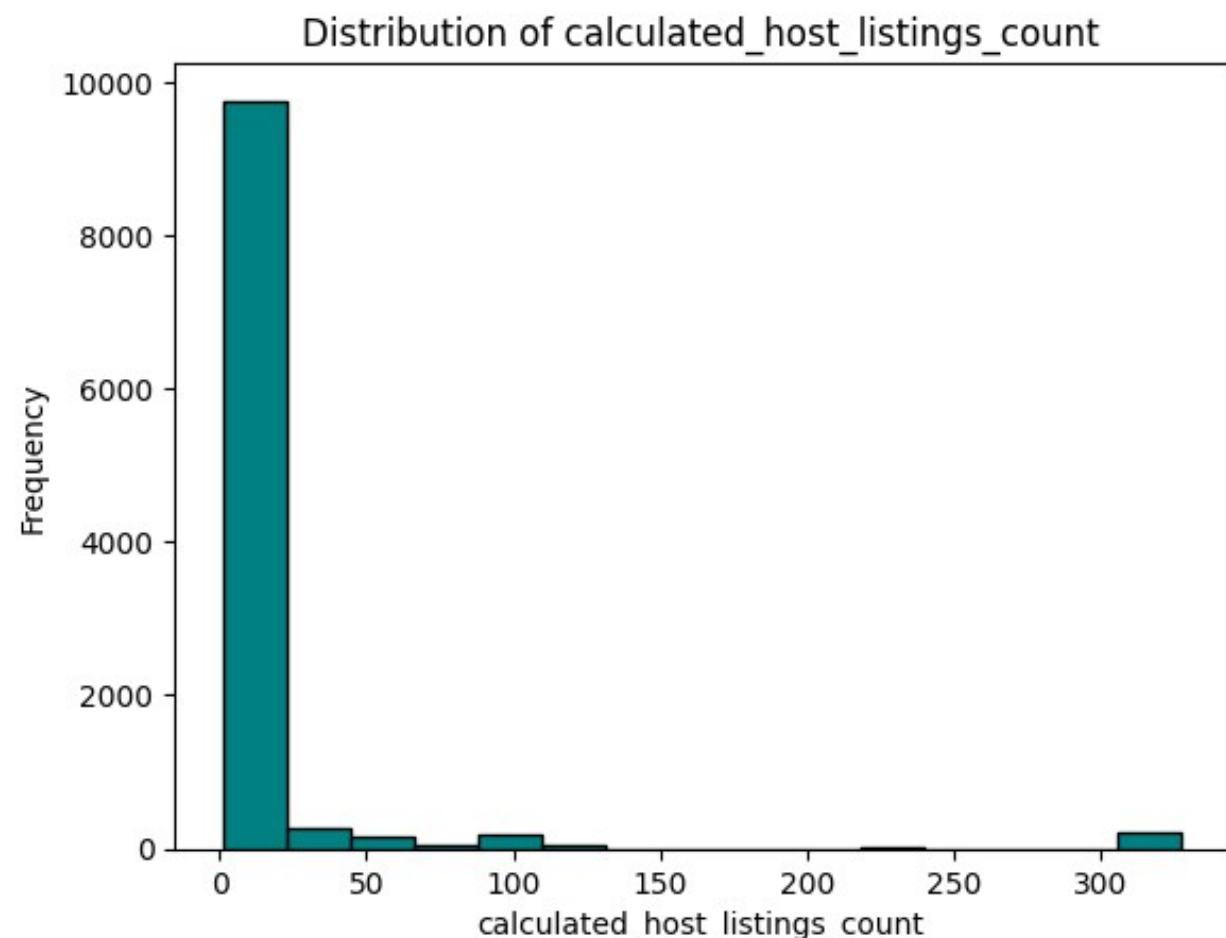
월별 리뷰 개수 분포 히스토그램

```
plt.hist(df[df['reviews_per_month'] < 30]['reviews_per_month'],  
         bins=30, edgecolor='black', color='khaki')  
plt.xlabel('reviews_per_month')  
plt.ylabel('Frequency')  
plt.title('Distribution of reviews_per_month')  
plt.show()
```



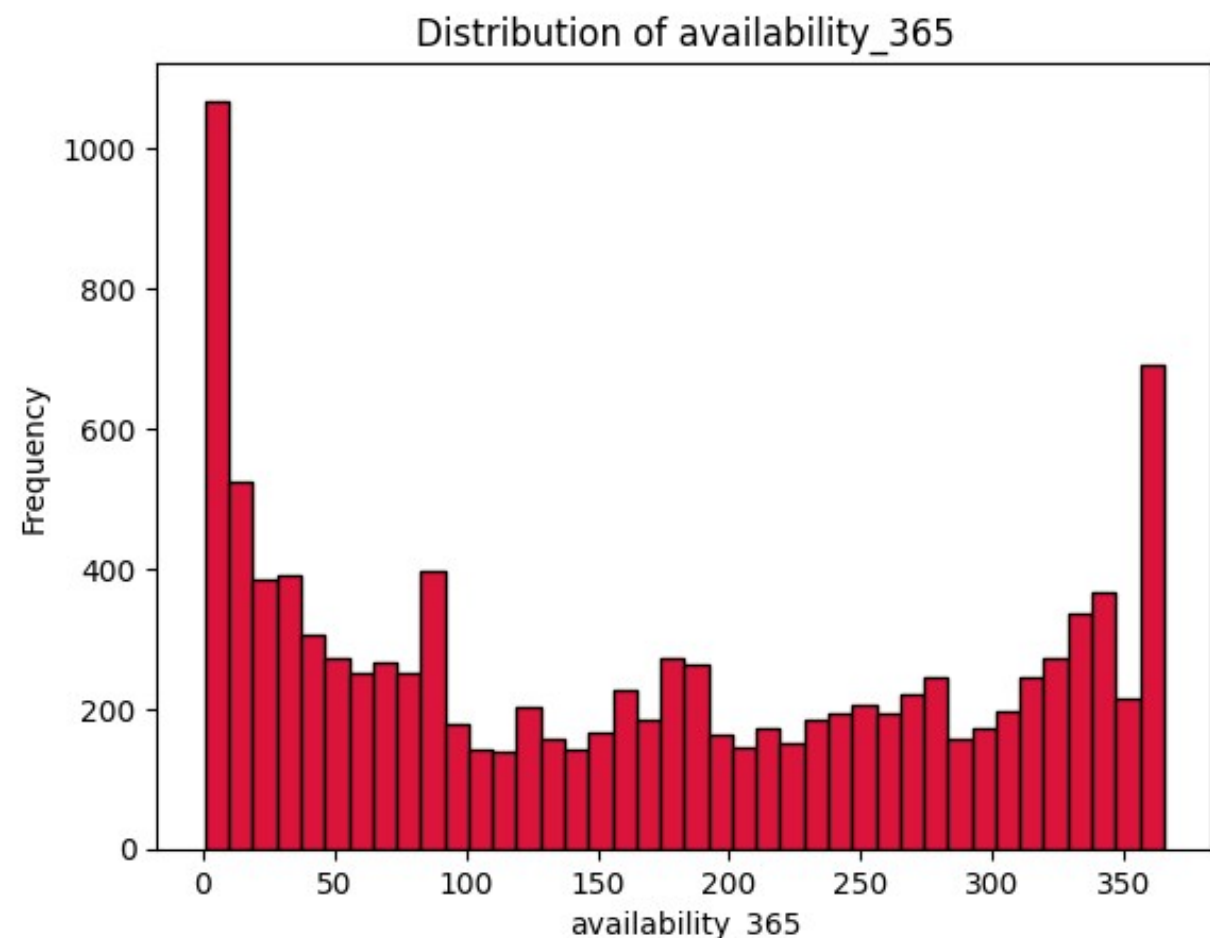
호스트가 운영 중인 객실 개수 분포 히스토그램

```
plt.hist(df['calculated_host_listings_count'], bins=15,  
         edgecolor='black', color='teal')  
plt.xlabel('calculated_host_listings_count')  
plt.ylabel('Frequency')  
plt.title('Distribution of calculated_host_listings_count')  
plt.show()
```



1년 중 예약 가능 일수 분포 히스토그램

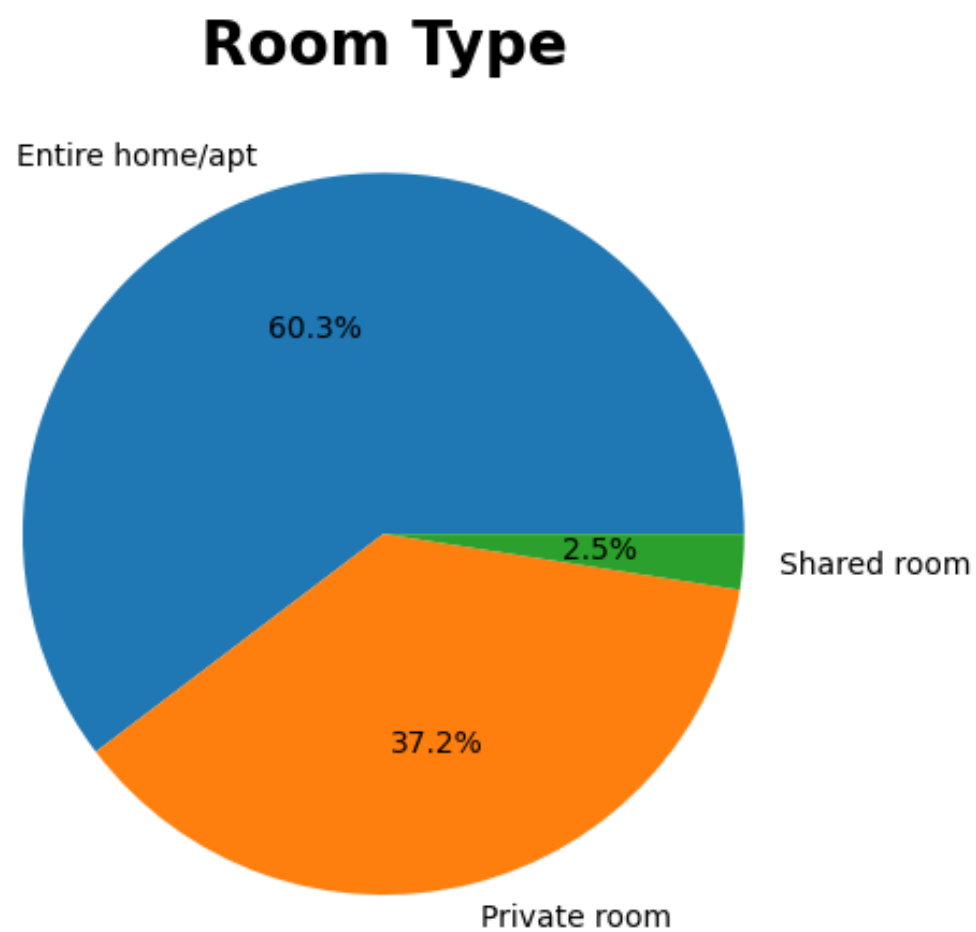
```
plt.hist(df['availability_365'], bins=40, edgecolor='black',  
         color='crimson')  
plt.xlabel('availability_365')  
plt.ylabel('Frequency')  
plt.title('Distribution of availability_365')  
plt.show()
```



범주형 변수 분포 시각화

Room Type 파이그래프

```
fig, ax8 = plt.subplots()
room_counts = df['room_type'].value_counts()
ax8.pie(room_counts, labels=room_counts.index, autopct='%1.1f%%')
ax8.set_title('Room Type', fontsize=20, fontweight='bold')
plt.tight_layout()
plt.show()
```



자치구 비율 파이그래프

'neighbourhood' 컬럼에 대해 각 값의 개수와 비율을 계산

```
neighbourhood_counts = df['neighbourhood'].value_counts()
neighbourhood_percentages = neighbourhood_counts / neighbourhood_counts.sum() * 100
```

3% 미만인 카테고리를 식별

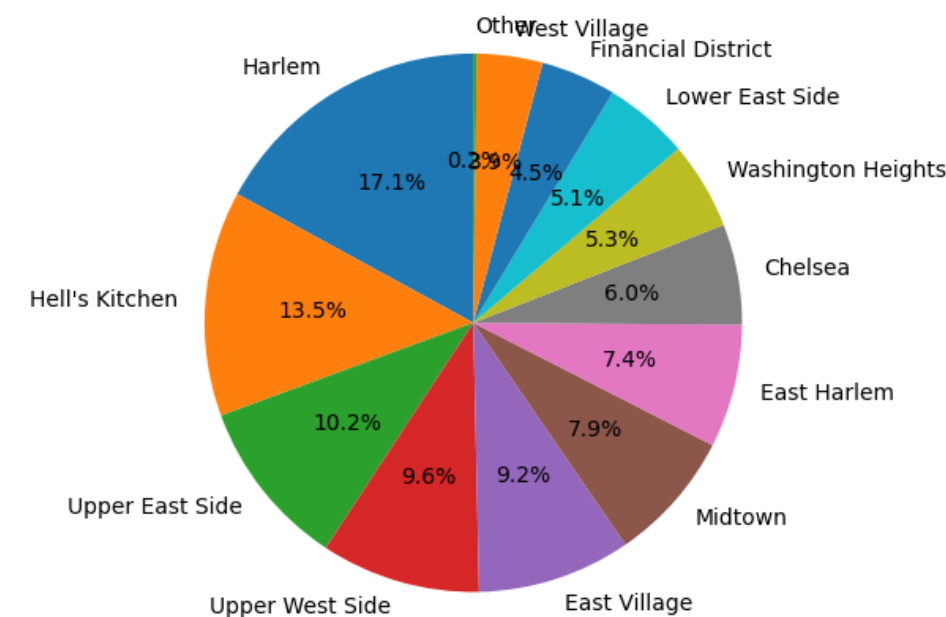
```
small_categories_nb = neighbourhood_percentages[neighbourhood_percentages < 3]
plot_data_nb = neighbourhood_counts[neighbourhood_percentages >= 3]
```

작은 비율의 카테고리 'Other' 항목 추가

```
if not small_categories_nb.empty:
    plot_data_nb['Other'] = small_categories_nb.sum()
```

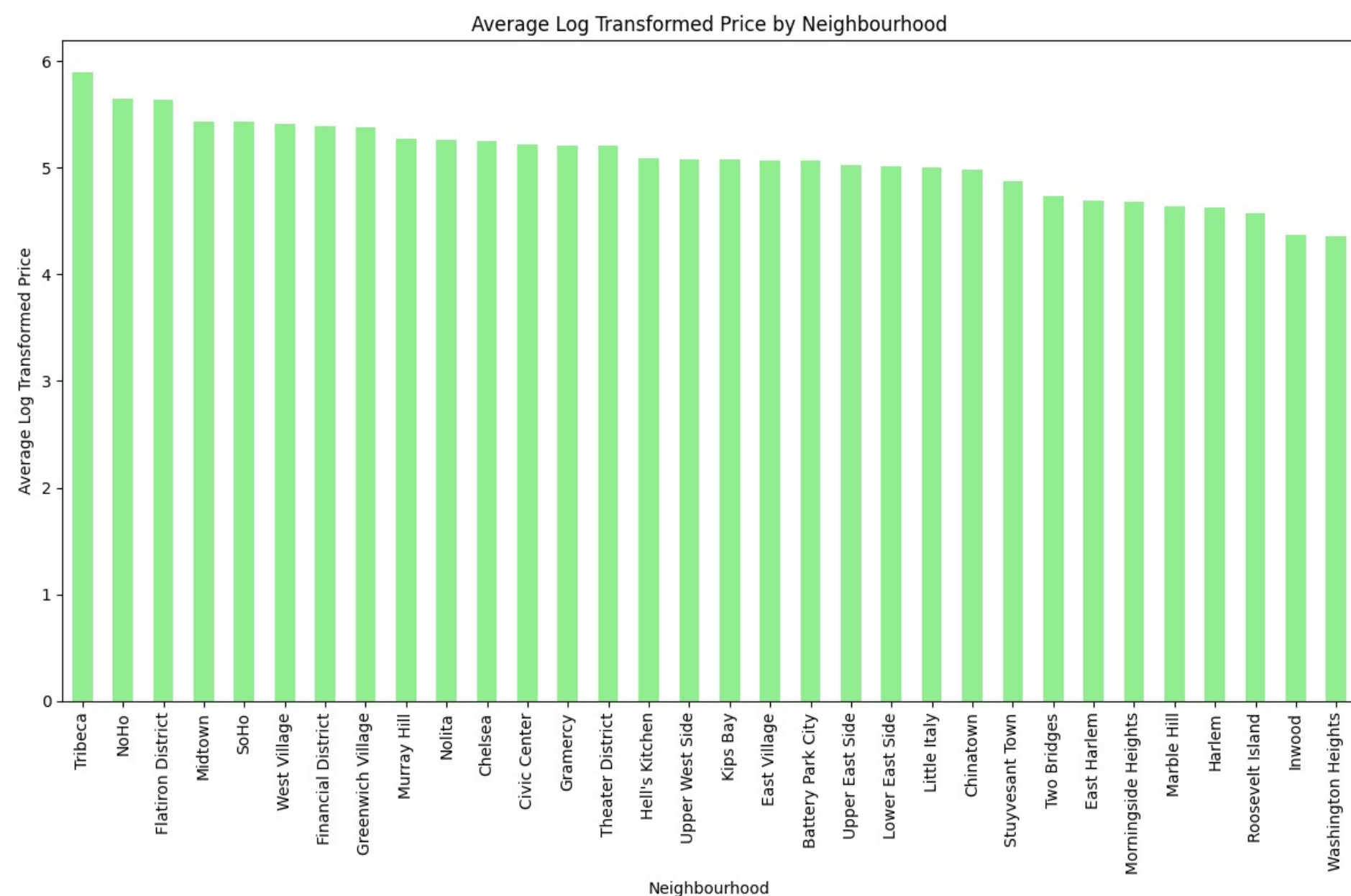
```
fig, ax = plt.subplots()
ax.pie(plot_data_nb, labels=plot_data_nb.index, autopct='%1.1f%', startangle=90)
ax.set_title('Neighbourhood Distribution (with "Other" for <3%)', fontsize=16, fontweight='bold')
plt.tight_layout()
plt.show()
```

Neighbourhood Distribution (with "Other" for <3%)



neighbourhood 별 평균 가격 그래프 - 가격 분포에 로그변환 사용

```
avg_price_log_by_neighbourhood = df.groupby('neighbourhood')['price_log'].mean().sort_values(ascending=False)
plt.figure(figsize=(12, 8))
avg_price_log_by_neighbourhood.plot(kind='bar', color='lightgreen')
plt.xlabel('Neighbourhood')
plt.ylabel('Average Log Transformed Price')
plt.title('Average Log Transformed Price by Neighbourhood')
plt.xticks(rotation=90)
plt.tight_layout()
plt.show()
```



사분위수 활용 후 이상치 제거

```
import numpy as np

Q1 = df['price_log'].quantile(0.25)
Q3 = df['price_log'].quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

df_filtered = df[(df['price_log'] >= lower_bound) & (df['price_log'] <= upper_bound)]

print(f"Original DataFrame shape: {df.shape}")
print(f"DataFrame shape after outlier removal: {df_filtered.shape}")
```

▼ Output

```
Original DataFrame shape: (10692, 16)
DataFrame shape after outlier removal: (10536, 16)
```

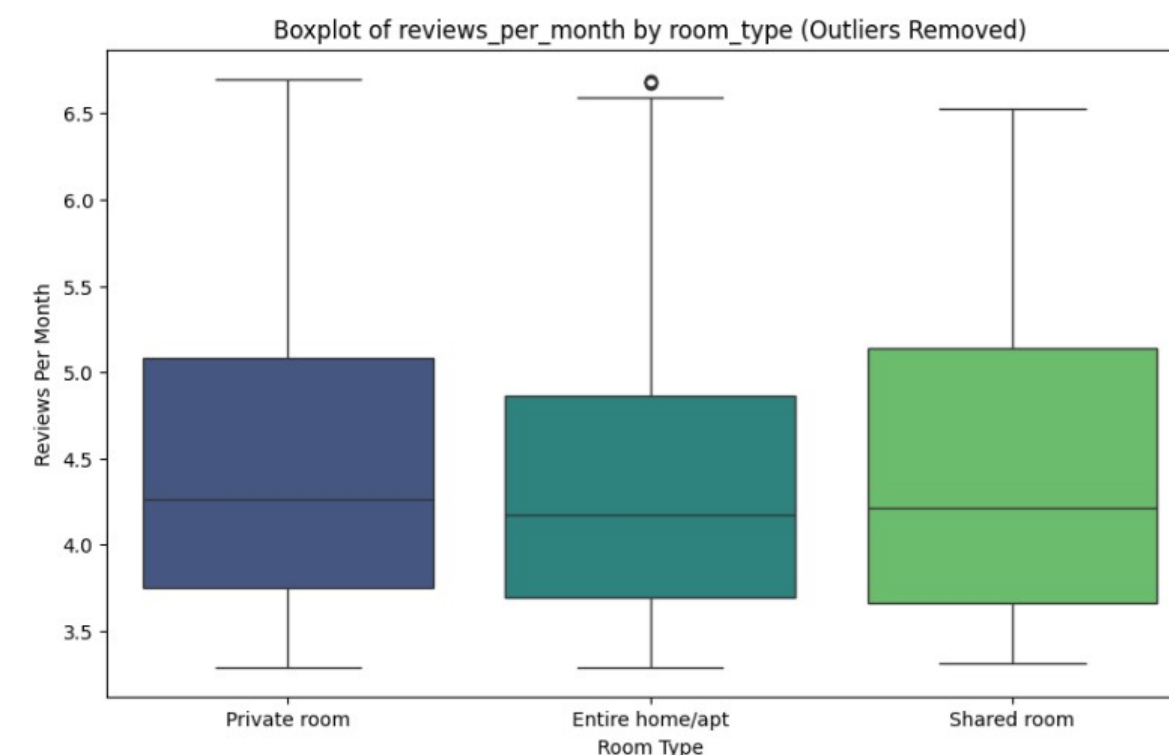
이상치 제거 후 Boxplot 재생성

필터링된 데이터프레임 생성

```
import seaborn as sns

df_filtered_outliers = df[(df['reviews_per_month'] >= lower_bound) & (df['reviews_per_month'] <= upper_bound)]

plt.figure(figsize=(10, 6))
sns.boxplot(data=df_filtered_outliers, x='room_type', y='reviews_per_month', hue='room_type', palette='viridis')
plt.title('Boxplot of reviews_per_month by room_type (Outliers Removed)')
plt.xlabel('Room Type')
plt.ylabel('Reviews Per Month')
plt.show()
```



객실 별 월별 리뷰 개수 평균값 및 개수 표

```
combined_room_type_stats = pd.merge(room_type_counts, average_reviews_by_room_type.rename('average_reviews_per_month'), on='room_type')
display(combined_room_type_stats)
```

	room_type	count	average_reviews_per_month
0	Entire home/apt	828	4.352560
1	Private room	749	4.483658
2	Shared room	59	4.488644

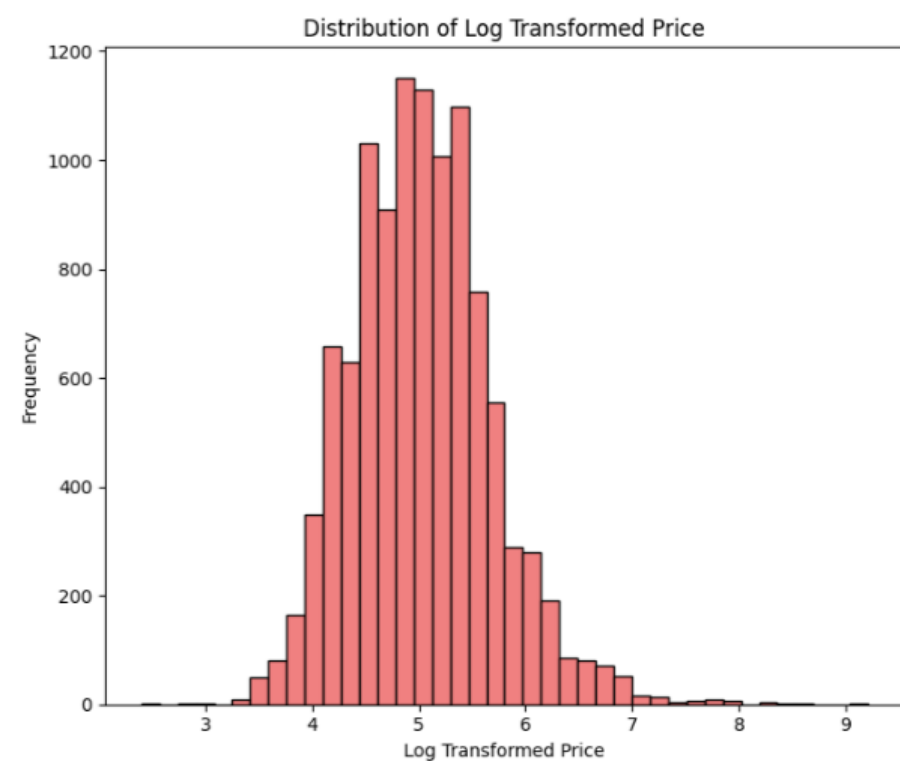
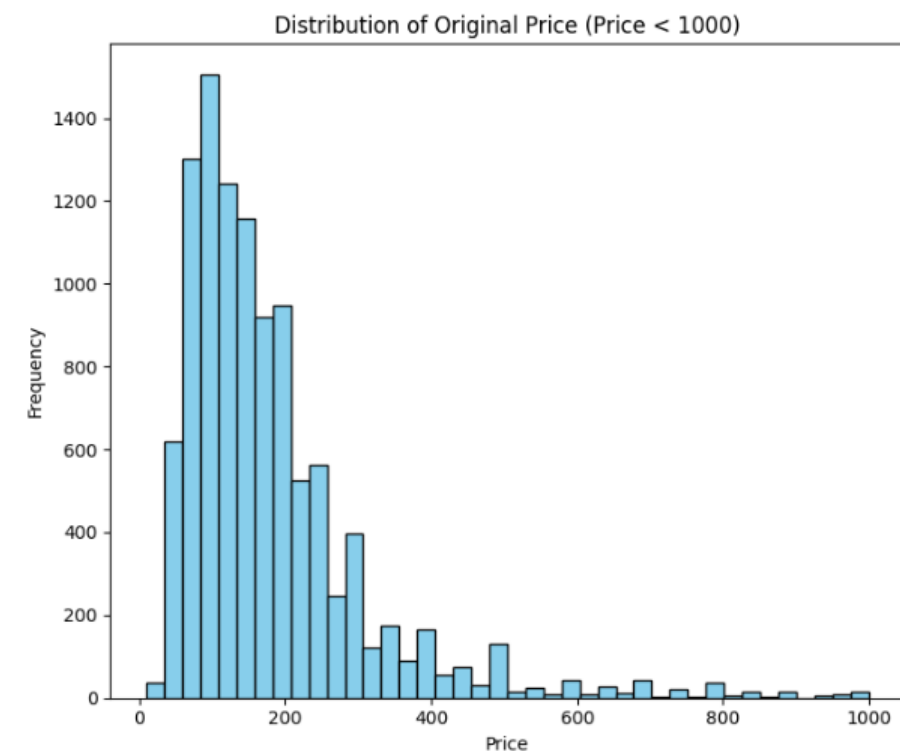
로그변환 전후 가격 분포 비교

```
fig, axes = plt.subplots(1, 2, figsize=(14, 6))
```

```
axes[0].hist(df[df['price'] < 1000]['price'], bins=40, edgecolor='black', color='skyblue')
axes[0].set_xlabel('Price')
axes[0].set_ylabel('Frequency')
axes[0].set_title('Distribution of Original Price (Price < 1000)')
```

```
axes[1].hist(df['price_log'], bins=40, edgecolor='black', color='lightcoral')
axes[1].set_xlabel('Log Transformed Price')
axes[1].set_ylabel('Frequency')
axes[1].set_title('Distribution of Log Transformed Price')
```

```
plt.tight_layout()
plt.show()
```



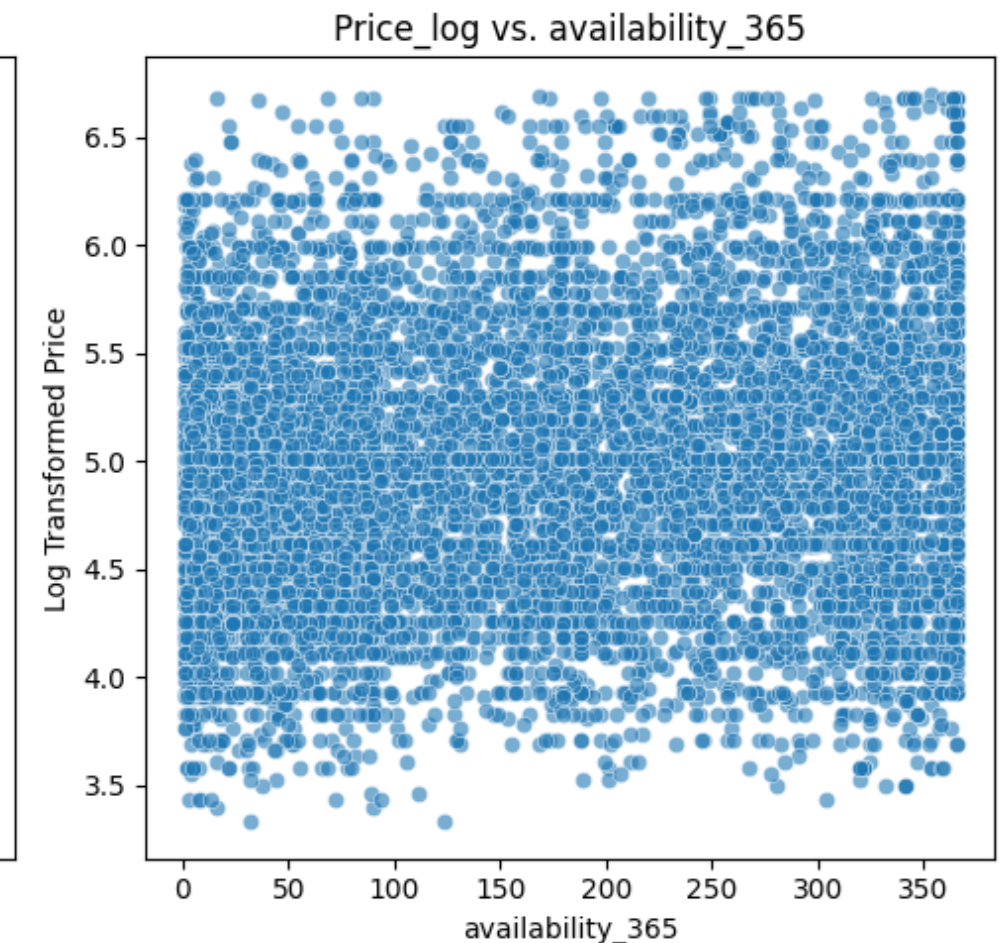
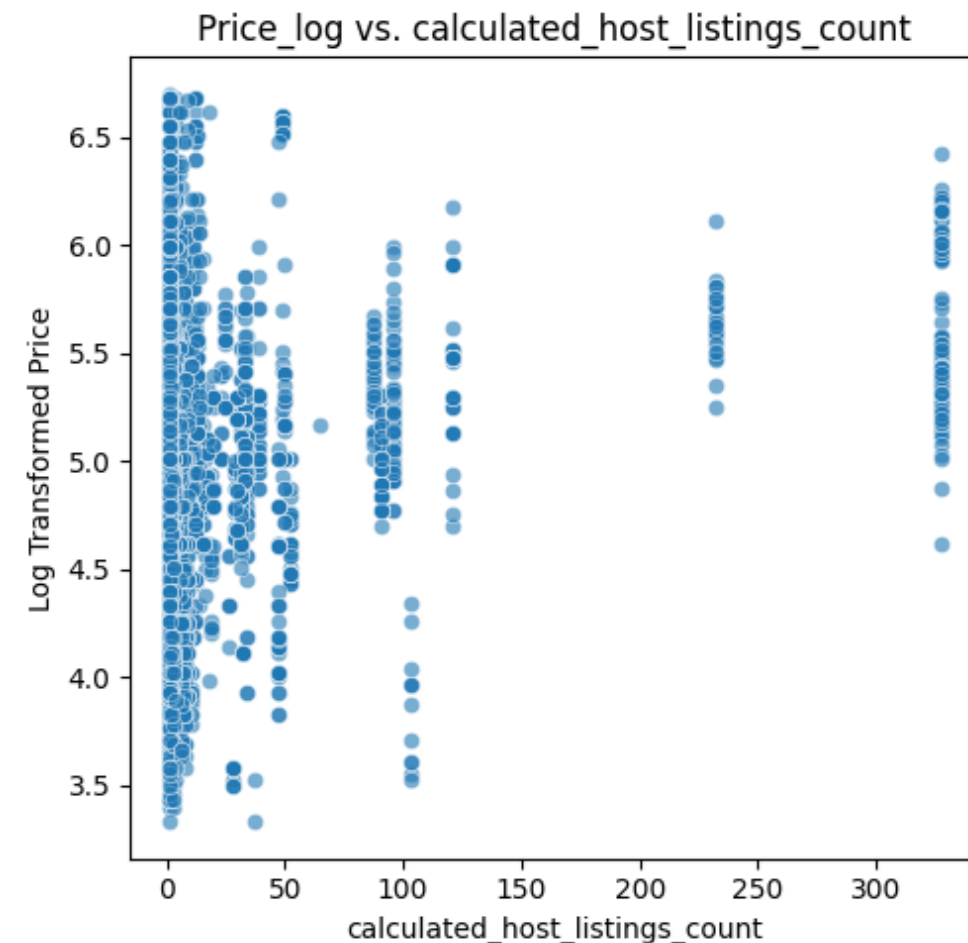
로그변환 후 가격 분포와 상관관계가 높은 변수들의 산점도

```
correlated_variables = top_3_correlated_to_price_log.index.tolist()

plt.figure(figsize=(15, 5))

for i, col in enumerate(correlated_variables):
    plt.subplot(1, 3, i + 1)
    sns.scatterplot(x=df_filtered[col], y=df_filtered['price_log'], alpha=0.6)
    plt.title(f'Price_log vs. {col}')
    plt.xlabel(col)
    plt.ylabel('Log Transformed Price')

plt.tight_layout()
plt.show()
```

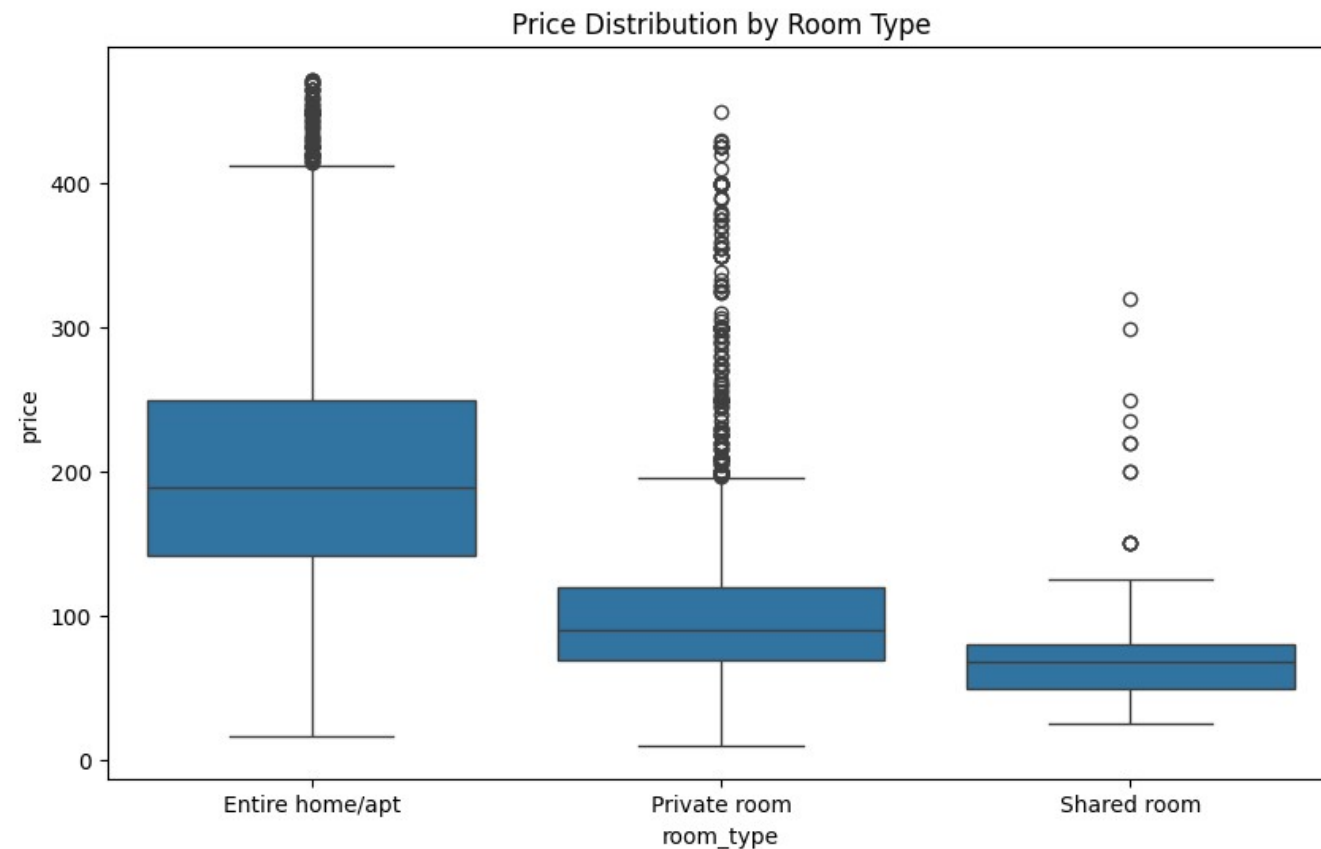


가격 변수와 다른 변수들과의 뚜렷한 관계가 보이지 않음
→ 비선형 관계의 가능성, 변수들과의 상호 작용이 있을 가능성 ↑

가격에 영향을 주는 변수 찾기

Room Type vs Price

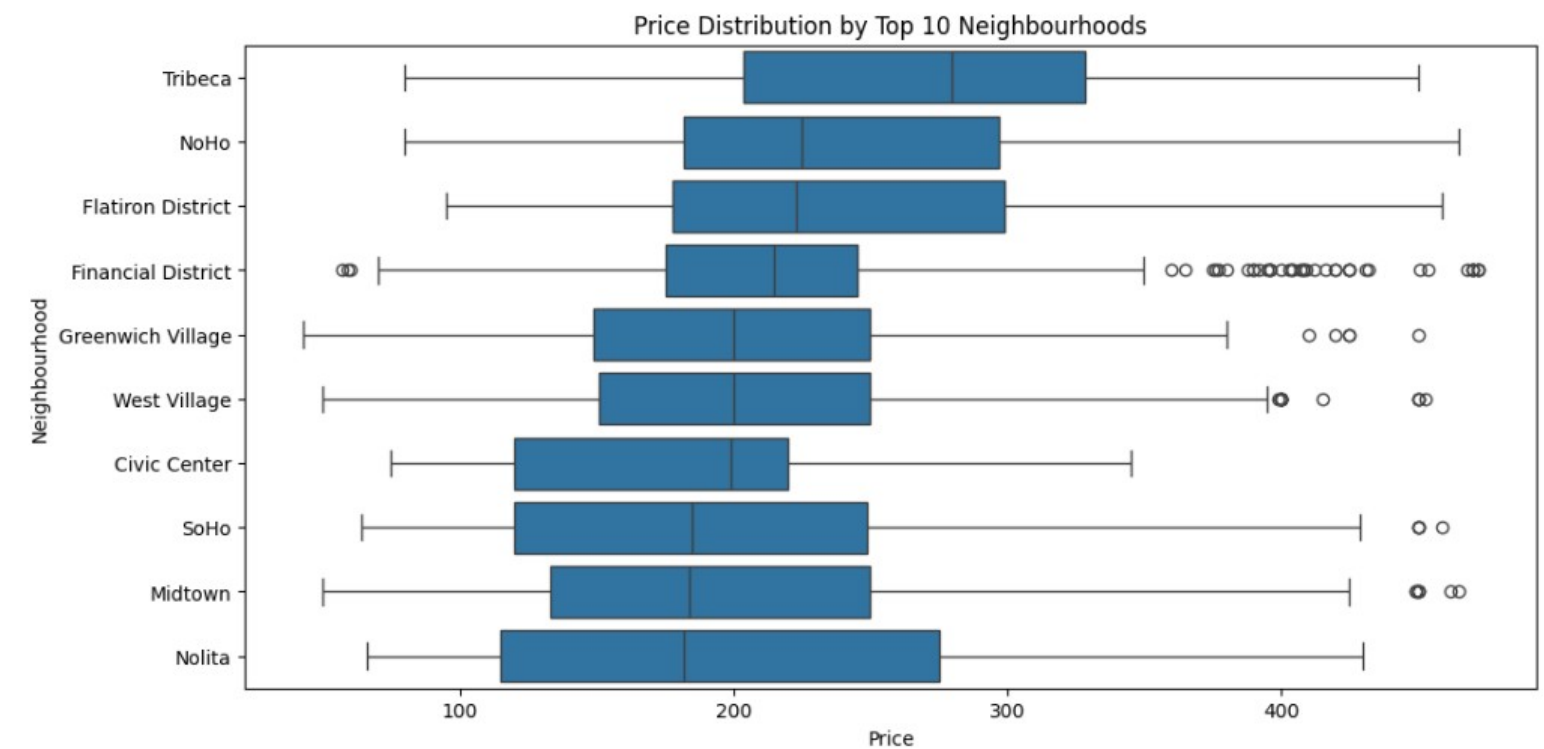
```
plt.figure(figsize=(10, 6))
sns.boxplot(x='room_type', y='price', data=df)
plt.title('Price Distribution by Room Type')
plt.show()
```



neighbourhood vs price (상위 10개 지역)

```
top_10_neigh = df.groupby('neighbourhood')['price'].median().sort_values(ascending=False).head(10).index
#이상치 영향을 줄이기 위해 평균 대신 중앙값 사용

plt.figure(figsize=(12, 6))
sns.boxplot(
    x='price',
    y='neighbourhood',
    data=df[df['neighbourhood'].isin(top_10_neigh)],
    order=top_10_neigh, # 중앙값 기준으로 내림차순 정렬
)
plt.title('Price Distribution by Top 10 Neighbourhoods')
plt.xlabel('Price')
plt.ylabel('Neighbourhood')
plt.show()
```



minimum_nights vs price

#산점도

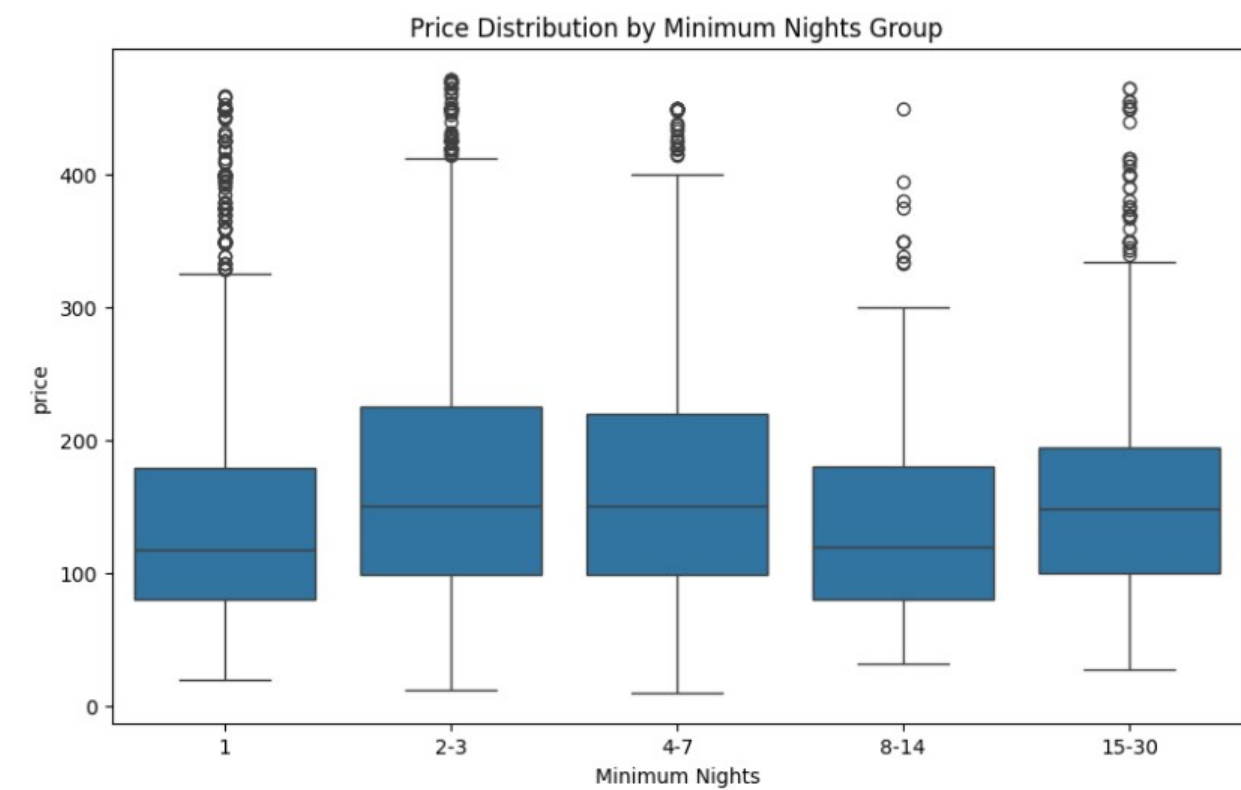
```
plt.figure(figsize=(10, 6))
sns.regplot(x='minimum_nights', y='price', data=df,
            scatter_kws={'alpha':0.3},
            fit_reg=False)

plt.title('Relationship between Minimum Nights and Price')
plt.xlim(0, 31) # 너무 긴 숙박일수는 제외하고 시각화
plt.show()
```

#구간별 상자그림

```
plt.figure(figsize=(10, 6))
df['nights_group'] = pd.cut(
    df['minimum_nights'],
    bins=[0,1,3,7,14,30],
    labels=['1', '2-3', '4-7', '8-14', '15-30']
)

sns.boxplot(x='nights_group', y='price', data=df)
plt.title('Price Distribution by Minimum Nights Group')
plt.xlabel('Minimum Nights')
plt.show()
```



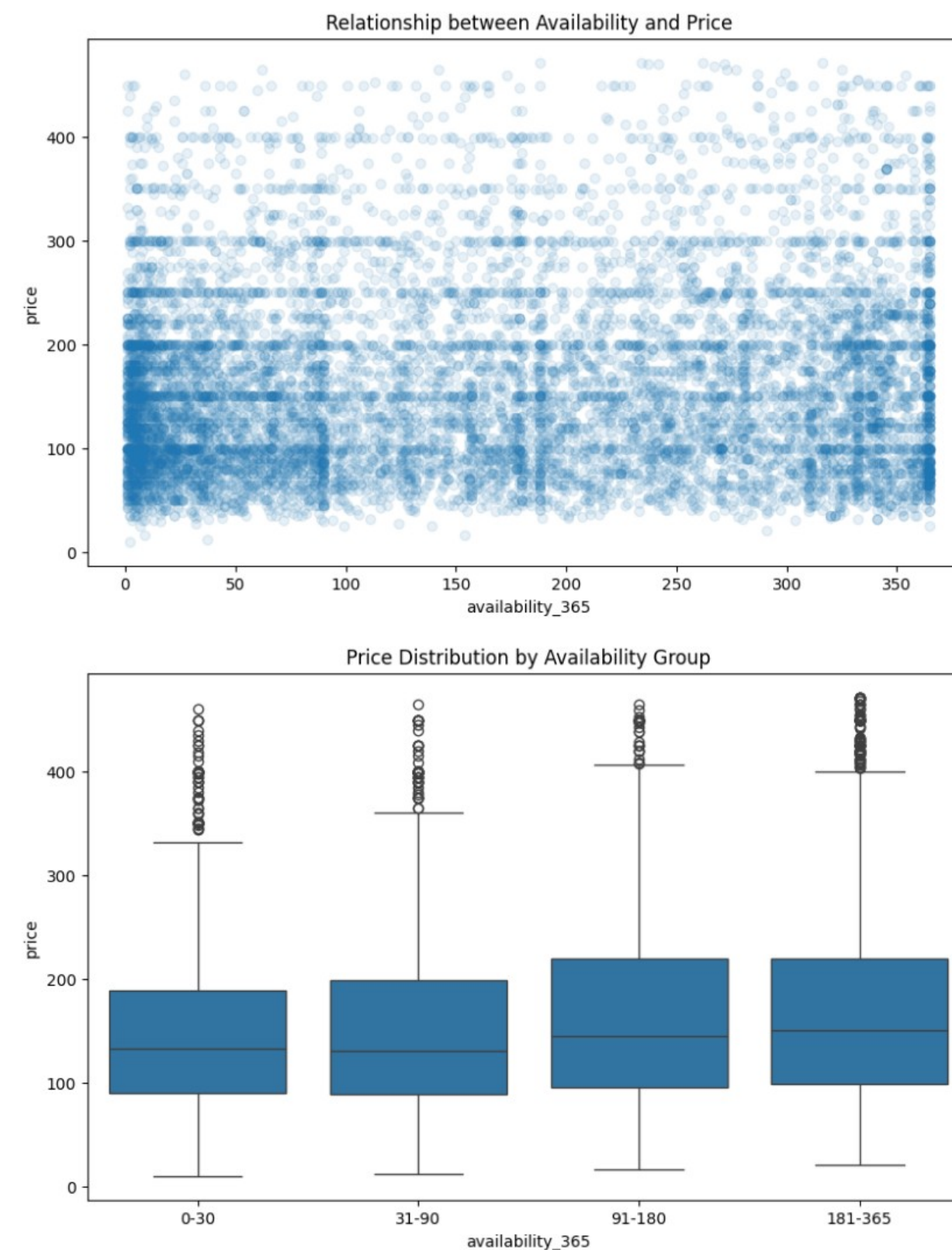
availability_365 vs price

#산점도

```
plt.figure(figsize=(10, 6))
sns.regplot(x='availability_365', y='price', data=df,
            scatter_kws={'alpha':0.1},
            fit_reg=False)
plt.title('Relationship between Availability and Price')
plt.show()
```

#구간별 상자그림

```
df['avail_group'] = pd.cut(
    df['availability_365'],
    bins=[0, 30, 90, 180, 365],
    labels=['0-30', '31-90', '91-180', '181-365']
)
plt.figure(figsize=(10, 6))
sns.boxplot(x='avail_group', y='price', data=df)
plt.xlabel('availability_365')
plt.title('Price Distribution by Availability Group')
plt.show()
```



Price vs Demand (Reviews per Month) by Room Type

#number_of_reviews는 누적 지표로 숙소 운영 기간의 영향을 받기 때문에

#수요를 파악하기 위해 reviews_per_month를 활용함

```
plt.figure(figsize=(10,6))
```

```
sns.scatterplot(
```

```
    x='price',
```

```
    y='reviews_per_month',
```

```
    hue='room_type',
```

```
    data=df,
```

```
    alpha=0.4
```

```
)
```

```
for rt in df['room_type'].unique():
```

```
    sns.regplot(
```

```
        x='price',
```

```
        y='reviews_per_month',
```

```
        data=df[df['room_type'] == rt],
```

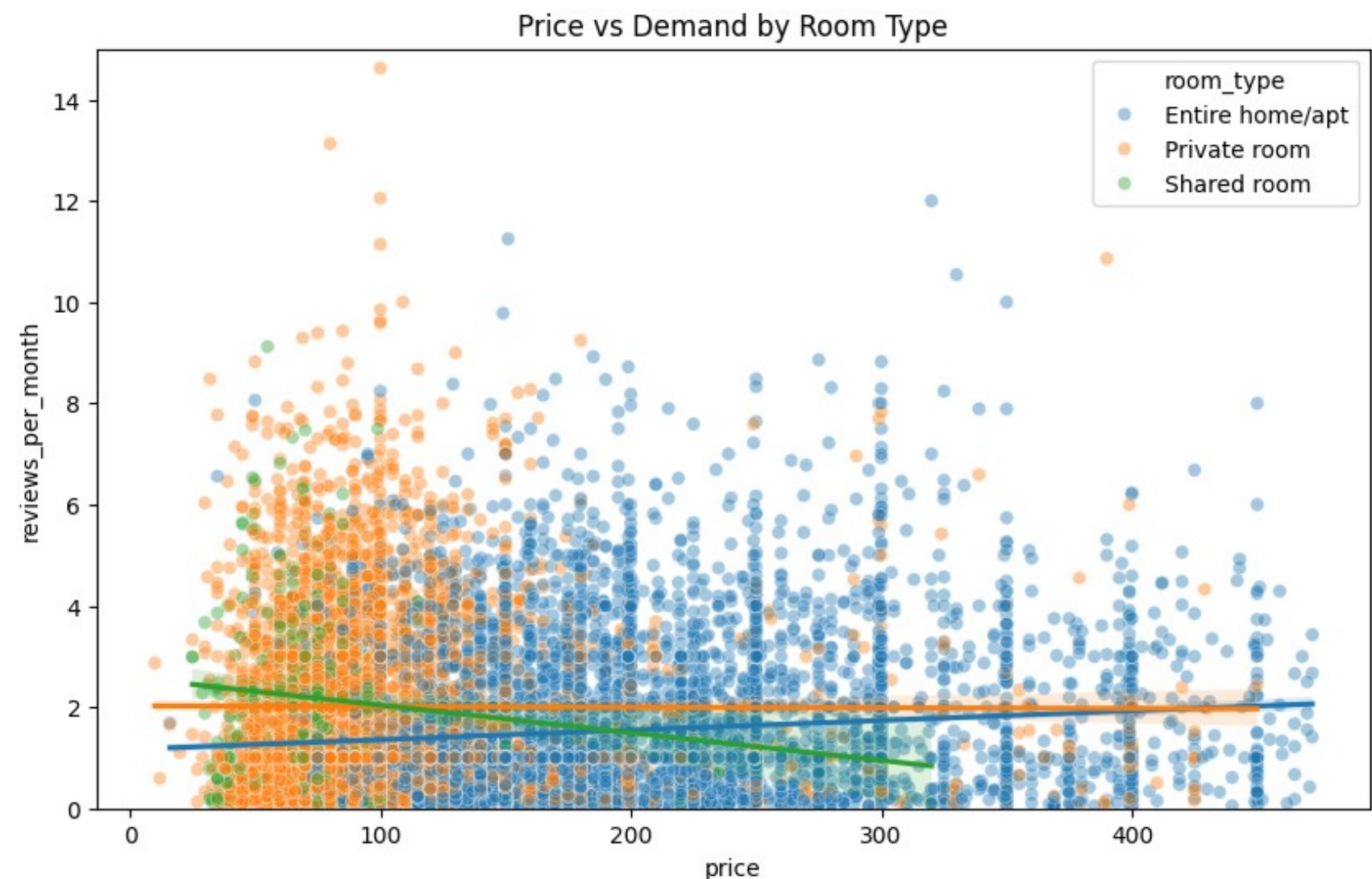
```
        scatter=False
```

```
    )
```

```
plt.title('Price vs Demand by Room Type')
```

```
plt.ylim(0, 15) #해석의 편의성을 위해 y축 범위 제한함
```

```
plt.show()
```



위치에 따른 가격 분포

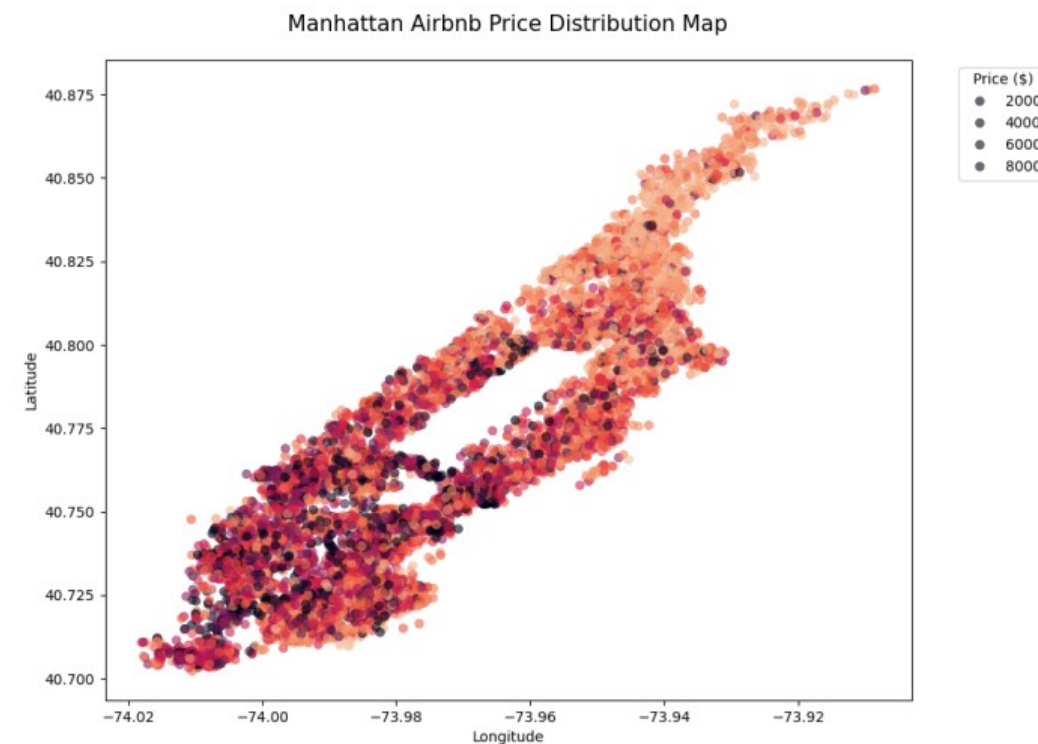
```
# 1. 그래프 크기 설정
plt.figure(figsize=(10, 8))

# 2. 산점도를 활용한 히트맵 생성
# x, y축을 경도/위도로 설정하고, 색상(hue)을 가격(price)으로 지정
scatter = sns.scatterplot(
    data=df,
    x='longitude',
    y='latitude',
    hue='price',
    palette='rocket_r', # 붉은색 계열의 팔레트
    alpha=0.6,         # 점의 투명도 (겹치는 부분 확인용)
    edgecolor=None,
    hue_norm=(0, 400)  # 가격 범위를 제한하여 시각적 차이를 극대화 (맨해튼 평균 고려)
)

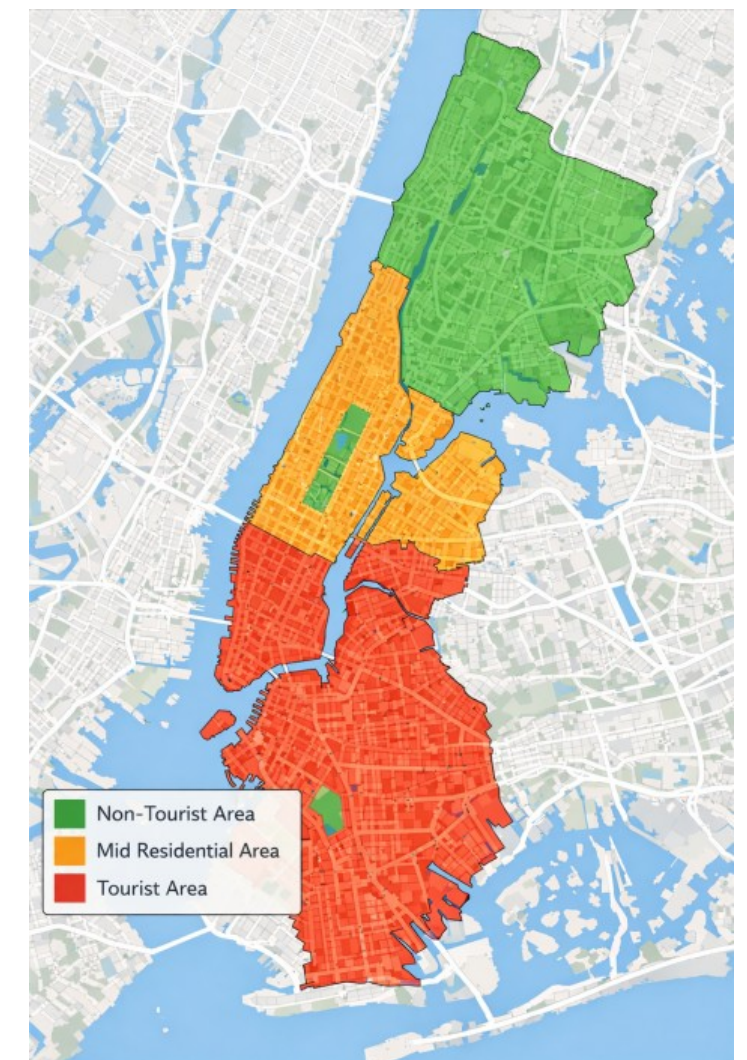
# 3. 제목 및 라벨 설정
plt.title('Manhattan Airbnb Price Distribution Map', fontsize=15, pad=20)
plt.xlabel('Longitude')
plt.ylabel('Latitude')

# 4. 범례 위치 조정
plt.legend(title='Price ($)', bbox_to_anchor=(1.05, 1), loc='upper left')

plt.show()
```



▼ 관광지, 비관광지, 준관광지 지도



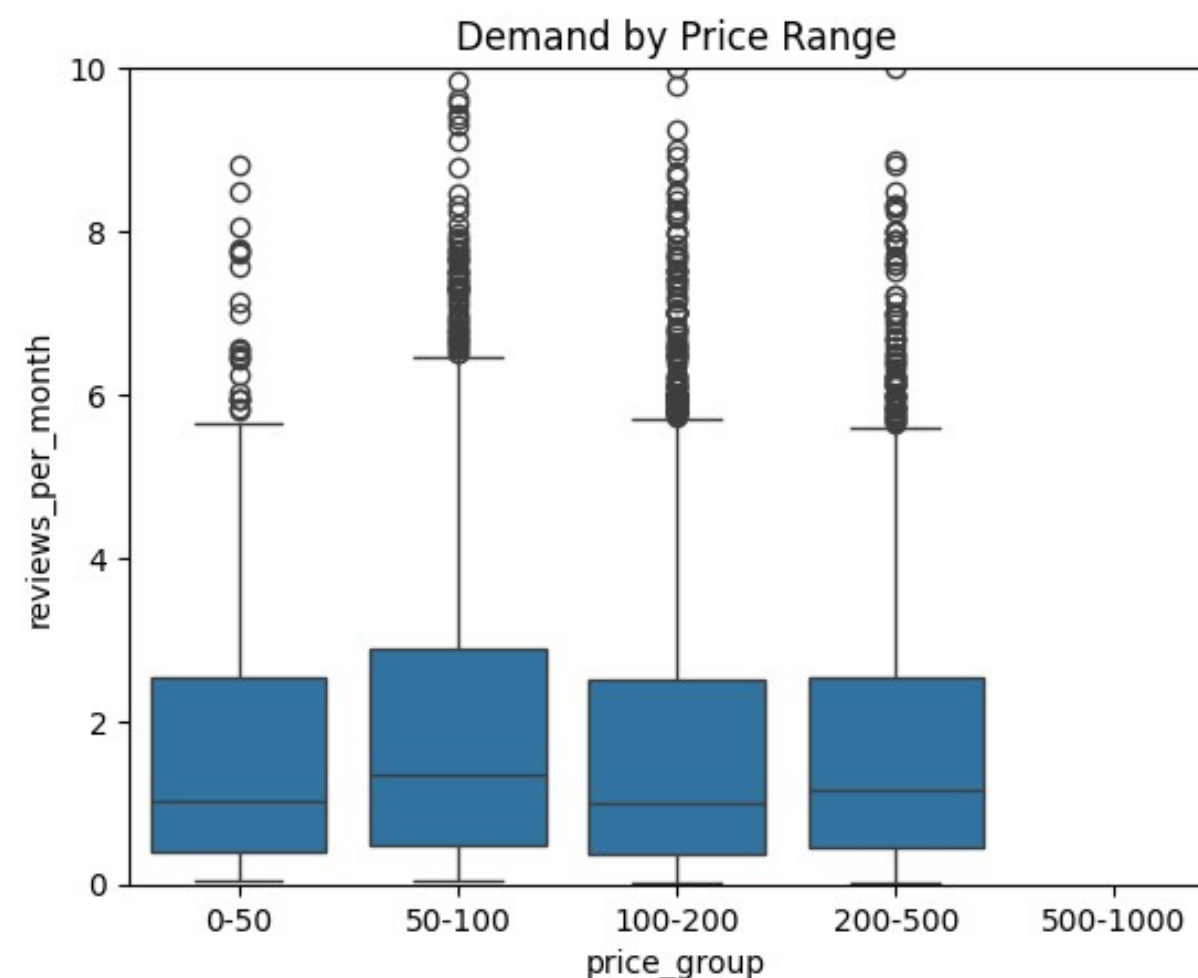
가성비 구간 찾기 (Demand vs Price)

```
#가성비 좋은 구간: 가격 낮고 수요(reviews_per_month) 높은 구간
df['price_group'] = pd.cut(
    df['price'],
    bins=[0,50,100,200,500,1000],
    labels=['0-50', '50-100', '100-200', '200-500', '500-1000']
)
```

```
sns.boxplot(x='price_group', y='reviews_per_month', data=df)
plt.ylim(0, 10) #해석의 편의성을 위해 y축 범위 제한함
plt.title('Demand by Price Range')
plt.show()
```

#가성비 그룹 선정을 위한 중앙값 출력 코드

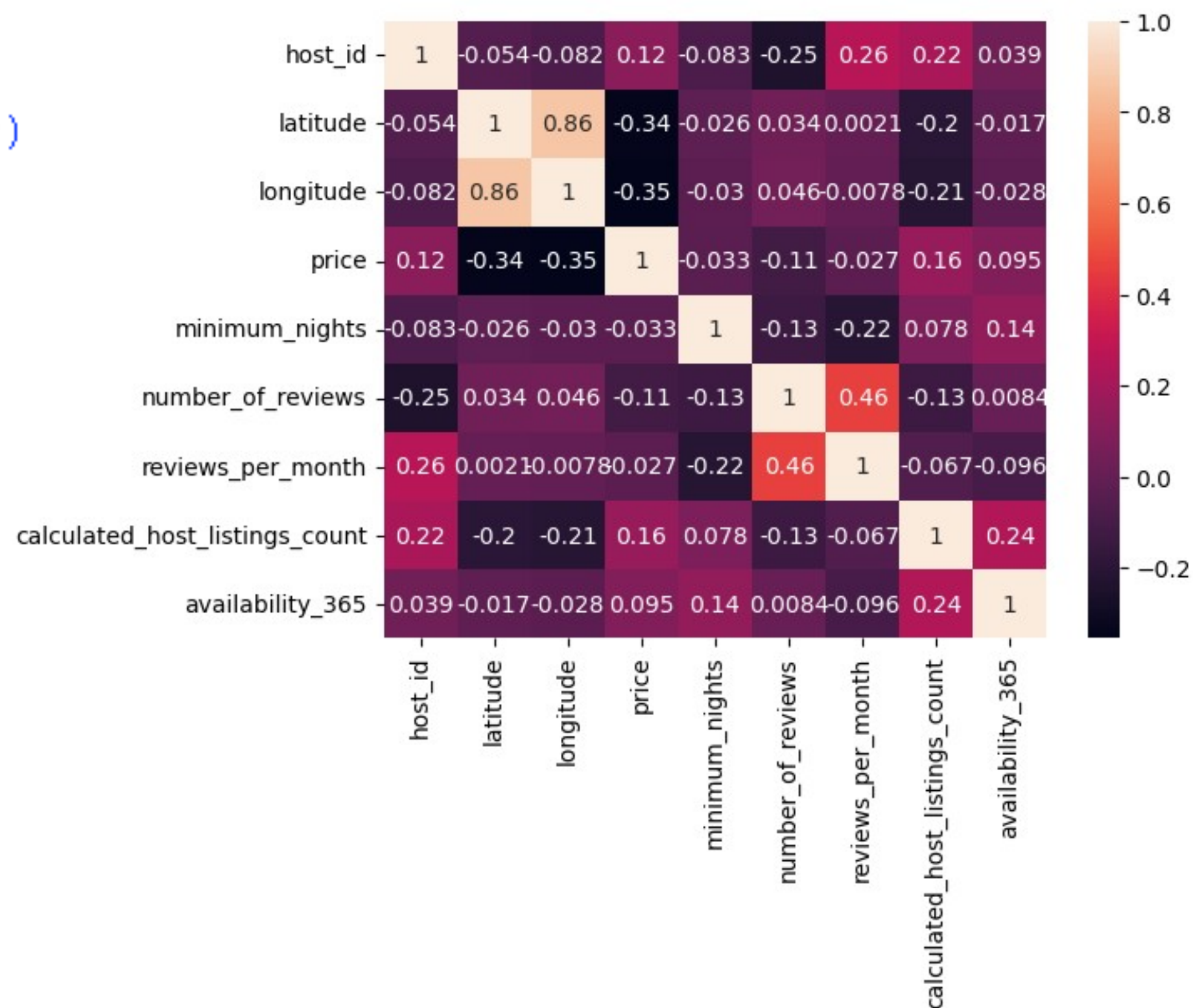
```
print(df.groupby('price_group')['reviews_per_month'].median())
```



```
price_group
0-50      1.02
50-100    1.35
100-200   1.00
200-500   1.15
500-1000  NaN
Name: reviews_per_month, dtype: float64
```

수치형 변수 간 상관분석(Heatmap)

```
numeric_df = df.select_dtypes(include=np.number)  
sns.heatmap(numeric_df.corr(), annot=True)
```



모델링

모델링 성능 비교 코드

```
from sklearn.model_selection import cross_val_predict
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from xgboost import XGBRegressor
from lightgbm import LGBMRegressor
```

데이터 준비

```
work_df = df.copy()

work_df['last_review'] = pd.to_datetime(work_df['last_review'])
work_df['days_since_last_review'] = (
    work_df['last_review'].max() - work_df['last_review']
).dt.days

features = [
    'room_type',
    'neighbourhood',
    'minimum_nights',
    'availability_365',
    'number_of_reviews',
    'reviews_per_month',
    'calculated_host_listings_count',
    'latitude',
    'longitude',
    'days_since_last_review'
]

X = pd.get_dummies(work_df[features], drop_first=True)
y = np.log1p(work_df['price'])
```

모델 정의

```
models = {
    "RandomForest": RandomForestRegressor(
        n_estimators=200,
        random_state=42,
        n_jobs=-1
    ),

    "GradientBoosting": GradientBoostingRegressor(
        n_estimators=200,
        learning_rate=0.05,
        max_depth=3,
        random_state=42
    ),

    "XGBoost": XGBRegressor(
        n_estimators=300,
        learning_rate=0.05,
        max_depth=6,
        subsample=0.8,
        colsample_bytree=0.8,
        random_state=42,
        n_jobs=-1
    ),

    "LightGBM": LGBMRegressor(
        n_estimators=300,
        learning_rate=0.05,
        max_depth=-1,
        random_state=42
    )
}
```

평가 함수

```
def evaluate_model(model, X, y):  
    y_pred_cv = cross_val_predict(model, X, y, cv=5)  
  
    y_true = np.expml(y)  
    y_pred = np.expml(y_pred_cv)  
  
    mae = mean_absolute_error(y_true, y_pred)  
    rmse = np.sqrt(mean_squared_error(y_true, y_pred))  
    r2 = r2_score(y_true, y_pred)  
  
    return mae, rmse, r2
```

모델별 성능 출력

```
results = []  
  
for name, model in models.items():  
    mae, rmse, r2 = evaluate_model(model, X, y)  
  
    results.append({  
        "Model": name,  
        "MAE": round(mae, 2),  
        "RMSE": round(rmse, 2),  
        "R2": round(r2, 4)  
    })  
  
results_df = pd.DataFrame(results).sort_values(by="R2", ascending=False)  
  
print(results_df)
```

▼ output

	Model	MAE	RMSE	R2
2	XGBoost	44.53	65.33	0.4543
3	LightGBM	44.51	65.33	0.4542
0	RandomForest	44.81	65.85	0.4456
1	GradientBoosting	44.64	66.09	0.4415

가성비 top5 함수

```
def recommend_top_by_value(work_df, top_n=5):
    import numpy as np
    import pandas as pd
    from xgboost import XGBRegressor
    from sklearn.model_selection import cross_val_predict

    result_df = work_df.copy()

    features = [
        'room_type',
        'neighbourhood',
        'minimum_nights',
        'availability_365',
        'number_of_reviews',
        'reviews_per_month',
        'calculated_host_listings_count',
        'latitude',
        'longitude',
        'days_since_last_review'
    ]

    X = pd.get_dummies(result_df[features], drop_first=True)
    y = np.log1p(result_df['price'])
```

```
X = pd.get_dummies(result_df[features], drop_first=True)
y = np.log1p(result_df['price'])

xgb_model = XGBRegressor(
    n_estimators=300,
    learning_rate=0.05,
    max_depth=6,
    subsample=0.8,
    colsample_bytree=0.8,
    random_state=42,
    n_jobs=-1
)

y_pred_cv = cross_val_predict(xgb_model, X, y, cv=5)

result_df['pred_price'] = np.expml(y_pred_cv)
result_df['value_score'] = (
    (result_df['pred_price'] - result_df['price']) / result_df['price']
)
```

* **value_score** : (예측 가격 - 실제 가격) / 실제 가격

- 실제 가격에 비해 가성비가 좋다고 판단될 수 있는 숙소를 찾는 지표
score 가 클수록 가성비가 좋다는 의미

가성비 top5 숙소 출력

```
top5 = recommend_top_by_value(work_df, top_n=5)
print(top5)
```

▼ output

	name	neighbourhood	room_type	price	pred_price	value_score	reviews_per_month	minimum_nights	availability_365	calculated_host_listings_count
0	Private Bedroom in West Village/Chelsea	Chelsea	Private room	12	123.467033	9.288919	0.59	3	37	1
1	Large 1br Duplex in Heart of Upper East Side	Upper East Side	Entire home/apt	16	151.341324	8.458833	1.69	2	9	1
2	Cozy feel at home studio	Kips Bay	Private room	10	89.461578	7.946158	2.87	5	2	1
3	The Oasis.	Harlem	Private room	16	69.442719	3.340170	1.66	2	154	3
4	BEST DEAL in NYC - Cute and Cozy Room Times Sq...	Hell's Kitchen	Private room	35	138.189636	2.948275	7.77	3	68	3

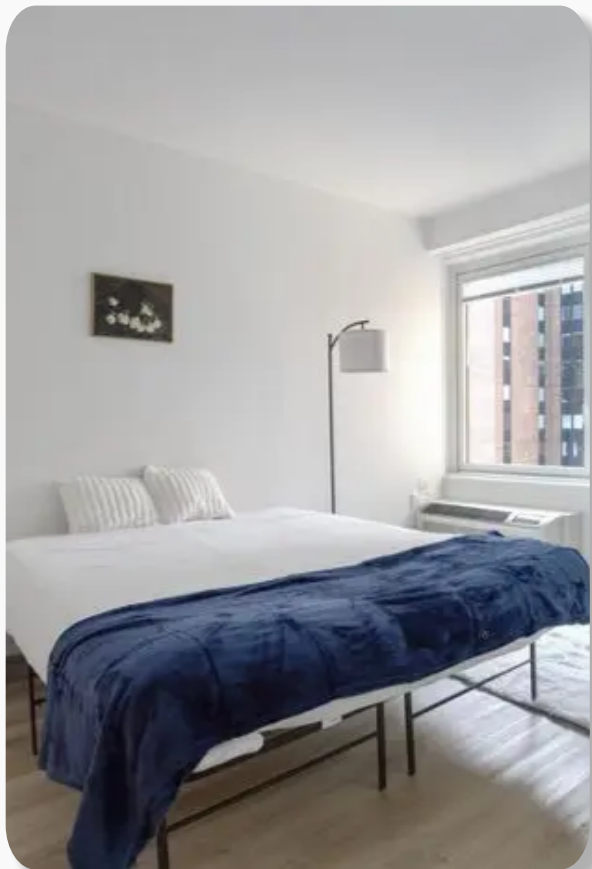
결론 도출 - 가성비 Top 5 숙소

(value_score 상위 5개 숙소)

Large 1br Duplex in Heart
of Upper East Side



Private Bedroom in West
Village/Chelsea



Cozy feel at home studio



The Oasis.



BEST DEAL in NYC -
Cute and Cozy Room Times Square

